

Travaux pratiques Informatique Embarquée

Université de Toulon
IUT GEII Première année



TPs Informatique Embarquée

- 1 Organisation
- 2 Le XIAO SAMD21 et la carte SmartLight
- 3 Travaux pratiques

1 Organisation

Séances de TPs - Groupe E - 2025/2026

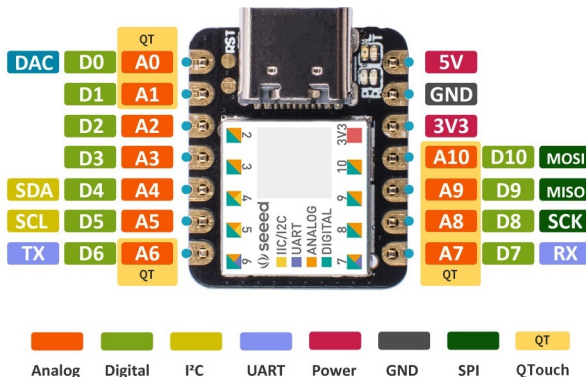
- Mardi aprèm
- 9 de 3h (13h30-16h30) et 2 de 1h30 (13h30-15h)
- Travail en monômes
- Vous pouvez vous aider et demander de l'aide !

Il y a des règles durant ces séances !

- Si pas de carte alors pas de TP !
- Pas de téléphone : éteint dans le sac
- Pas d'IA durant les séances
- Lors de l'évaluation, vous n'aurez pas accès aux outils d'IA

2 Le XIAO SAMD21 et la carte SmartLight

- Le XIAO SAMD21
- La carte SMARTLIGHT
- La carte SMARTLIGHT - Déclaration de broches
- Programmation - Le Main
- Programmation - Les entrées/sorties numériques
- Programmation - Les entrées/sorties analogiques
- Programmation - La fonction millis()
- Programmation - Système temps réel



A0..A10 : Entrées analogiques

D0..D10 : Entrées ou sorties numériques

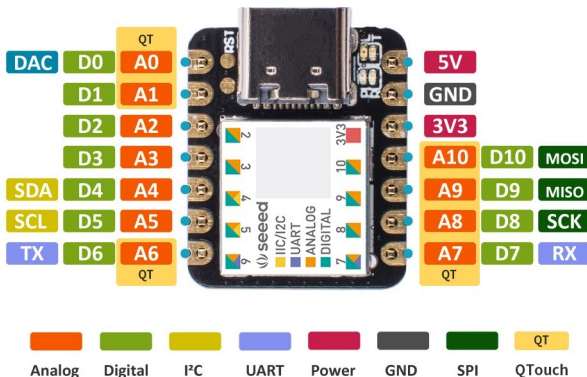
DAC : Sortie analogique

Bus série :

- **I²C** (Inter-Integrated Circuit - **SDA** et **SCL**)
- **UART** (Universal Asynchronous Receiver Transmitter - **TX** et **RX**)
- **SPI** (Serial Peripheral Interface - **MOSI**, **MISO** et **SCK**)

QTouch : Capacitive Touch Sensor - Peripheral Touch Controller (PTC)

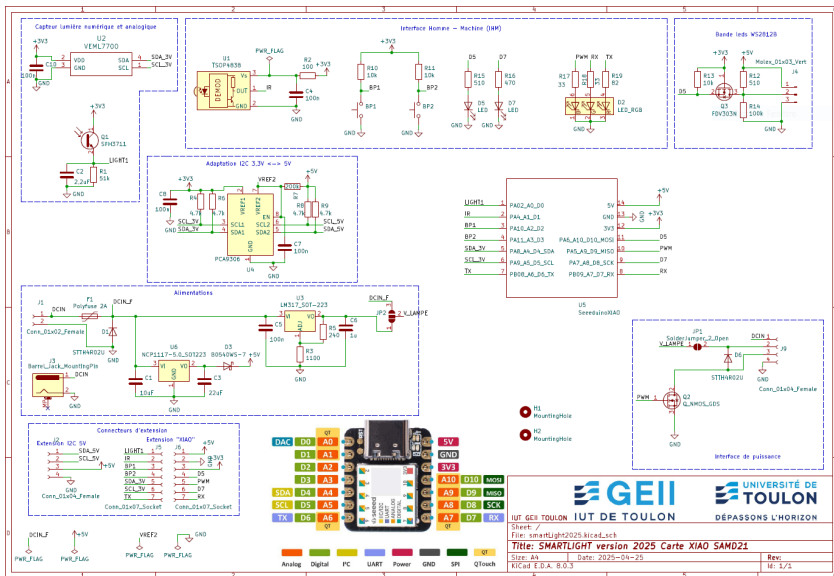
Le XIAO SAMD21



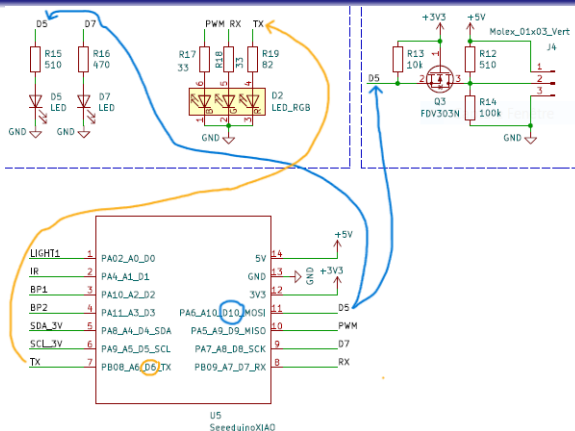
Conclusion : Entrée-Sortie à usage général

- Une même broche de la puce peut avoir différentes fonctions
- Jamais plus d'une fonction au même moment !
- General Purpose Input Output (GPIO)

La carte SMARTLIGHT



La carte SMARTLIGHT - Déclaration de broches



```

1 #define LED_D5_PIN D10 // Et non #define LED_D5_PIN D5 !
2 #define LED_RED_PIN D6 // Et non #define LED_RED_PIN TX !
3 #define LED_D7_PIN D8 // ...
4 #define LED_GREEN_PIN D7

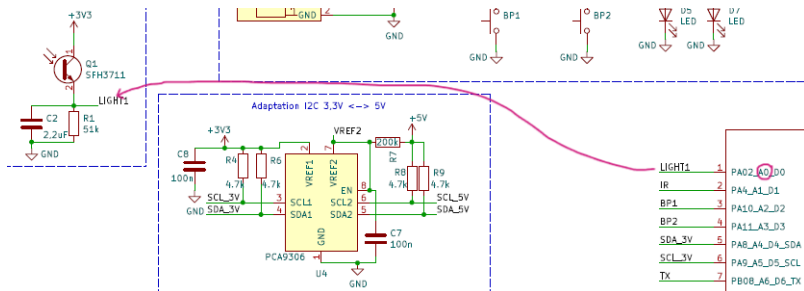
```

Exemples de déclaration de broches - Sorties numériques



2

La carte SMARTLIGHT - Déclaration de broches



1

```
#define LIGHT1_PIN A0
```

Exemples de déclaration de broches - Entrée analogique

```
#include <Arduino.h>

// Declared weak in Arduino.h to allow user redefinitions.
int atexit(void (* /*func*/ )()) { return 0; }

// Weak empty variant initialization function.
// May be redefined by variant files.
void initVariant() __attribute__((weak));
void initVariant() { }

void setupUSB() __attribute__((weak));
void setupUSB() { }

int main(void)
{
    init();

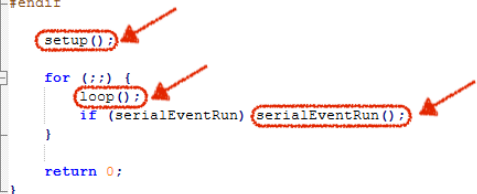
    initVariant();

#if defined(USBCON)
    USBDevice.attach();
#endif

    setup();

    for (;;) {
        loop();
        if (serialEventRun) serialEventRun();
    }

    return 0;
}
```



La fonction **loop()**

- Le nom **loop()** est mal choisi !
- C'est un "faux ami"
- C'est une fonction pas une boucle !
- Mais cette fonction est bien appelée dans une boucle **for(;;)**
- Et cette boucle n'a pas de fin d'exécution : boucle infinie !

[Lien Arduino language Reference](#)

digitalWrite()

Last revision • 23/04/2025

Description

Write a `HIGH` or a `LOW` value to a digital pin.

If the pin has been configured as an `OUTPUT` with `pinMode()`, its voltage will be set to the corresponding value: 5V (or 3.3V on 3.3V boards) for `HIGH` and 0V (ground) for `LOW`.

If the pin is configured as an `INPUT`, `digitalWrite()` will enable (`HIGH`) or disable (`LOW`) the **internal pull-up** on the input pin. It is recommended to set the `pinMode()` to `INPUT_PULLUP` to enable the internal pull-up resistor. See the [Digital Pins](#) tutorial for more information.

If you do not set the pin as an `OUTPUT`, and connect an LED to it, when calling `digitalWrite(pin, HIGH)`, the LED may appear dim. Without explicitly setting `pinMode()`, `digitalWrite()` will have enabled the internal pull-up resistor, which acts like a large current-limiting resistor.

digitalRead()

Last revision • 23/04/2025

Description

Reads the value from a specified digital pin, either `HIGH` or `LOW`.

Syntax

Use the following function to read the value of a digital pin:

```
digitalRead(pin)
```

analogWrite()

Last revision - 09/05/2025

Description

Writes an analog value (**PWM wave**) to a pin. Can be used to light a LED at varying brightness or drive a motor at various speeds. After a call to `analogWrite()`, the pin will generate a steady rectangular wave of the specified duty cycle until the next call to `analogWrite()` (or a call to `digitalRead()` or `digitalWrite()` on the same pin.

Check your board pinout to know which are the officially supported PWM pins. While some boards have additional pins capable of PWM, using them is recommended only for advanced users that can account for timer availability and potential conflicts with other uses of those pins.

In addition to PWM capabilities some boards have true analog output when using `analogWrite()` on the `DAC` marked pins. Check your board pinout to find out if the DAC is available.

analogRead()

Last revision - 09/05/2025

Description

Reads the value from a specified analog input pin.

An **Arduino UNO** for example, contains a multichannel, **10-bit analog to digital converter (ADC)**. This means that it will map input voltages between 0 and the operating voltage **(+5 VDC)** into integer values between **0 and 1023**. This yields a resolution between readings of: 5 volts / 1024 units or **0.0049 volts (4.9 mV) per unit**.

The voltage input range can be changed using **analogReference()**. The default **analogRead()** resolution on Arduino boards is set to 10 bits, for compatibility. You need to use **analogReadResolution()** to change it to a higher resolution.

millis()

Last revision • 05/06/2025

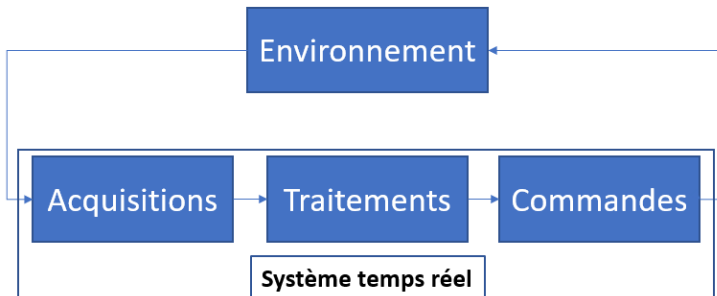
Description

Returns the number of milliseconds passed since the Arduino board began running the current program. This number will overflow (go back to zero), after approximately 50 days.

Syntax

Use the following function to get the exact time the board has been running the current program in milliseconds:

```
millis()
```



3 Travaux pratiques

- Bonnes pratiques de programmation 1 : la casse chameau !
- Bonnes pratiques de programmation 2 : ne vous répétez pas !
- TP1 - Le problème des boutons poussoirs
- TP1 - Détection de front sur signal numérique
- Exemple général de GRAFCET
- TP1 - GRAFCET 10, 11 et 12
- TP2 - La relation $E_v = f(n)$
- TP2 - GRAFCET 4

[Lien Wikipedia](#)



```
1 #define LIGHT1_PIN A0 // Constantes tjrs en majuscules
2 int redLed, bp1, lastBp1;
3 void unNomDeFonction(int paramNum1, char paramNum2) {
4     //...
5 }
```

Exemples de notation en casse chameau : variables et fonctions

```

1 // ...
2 void loop() {
3     // ...
4     blinkRedLed();
5     blinkBlueLed();
6     // ...
7 }
8 void blinkRedLed() {
9     compute = time % T_LED_RED;
10    if(compute < T_LED_RED/2) digitalWrite(LED_RED_PIN,
        HIGH);
11    else digitalWrite(LED_RED_PIN, LOW);
12 }
13 void blinkBlueLed() {
14     compute = time % T_LED_BLUE;
15    if(compute < T_LED_BLUE/2) digitalWrite(LED_BLUE_PIN,
        HIGH);
16    else digitalWrite(LED_BLUE_PIN, LOW);
17 }

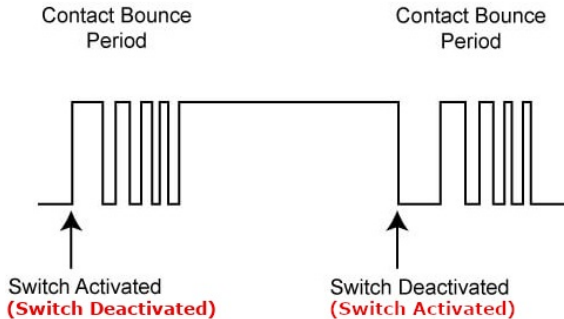
```

Exemple de répétition : il faut factoriser le code de clignotement

```
1 // ...
2 void loop() {
3     // ...
4     blinkLed(LED_RED_PIN, T_LED_RED);
5     blinkLed(LED_BLUE_PIN, T_LED_BLUE);
6     // ...
7 }
8 void blinkLed(int ledPin, int period) {
9     compute = time % period;
10    if(compute < period/2) digitalWrite(ledPin, HIGH);
11    else digitalWrite(ledPin, LOW);
12 }
```

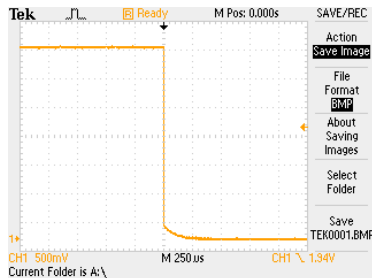
Exemple de répétition : factorisation effectuée

TP1 - Le problème des boutons poussoirs



Suivant le montage électrique : pull-up ou pull-down
pull-up -> résistance de rappel à l'état haut
pull-down -> résistance de rappel à l'état bas
Le montage est en rappel à l'état haut sur la carte

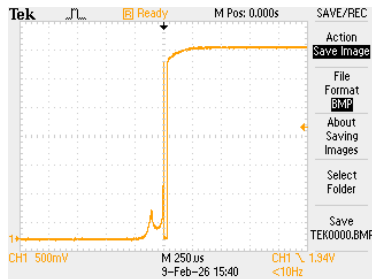
Appuie sur le bouton poussoir



Mise en contact des lames métalliques :

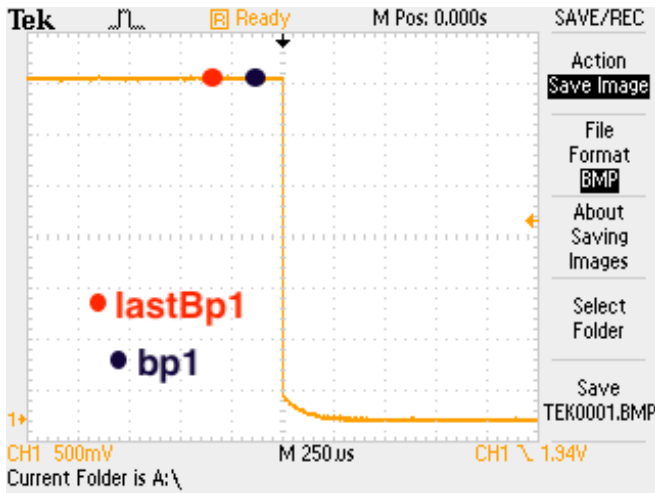
- Le doigt plaque et maintient les lames métalliques en contact
- D'un point de vue mécanique, il n'y a pas de rebonds...
- ... ou en dessous du niveau logique haut

Relâchement du bouton poussoir



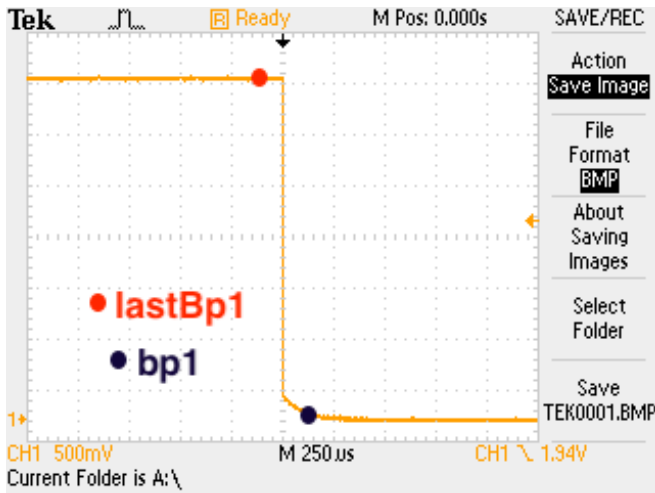
Déconnexion des lames métalliques :

- Le doigt libère une des lames métalliques
- D'un point de vue mécanique, les rebonds sont très probables
- Niveau logique bas retrouvé lors des rebonds

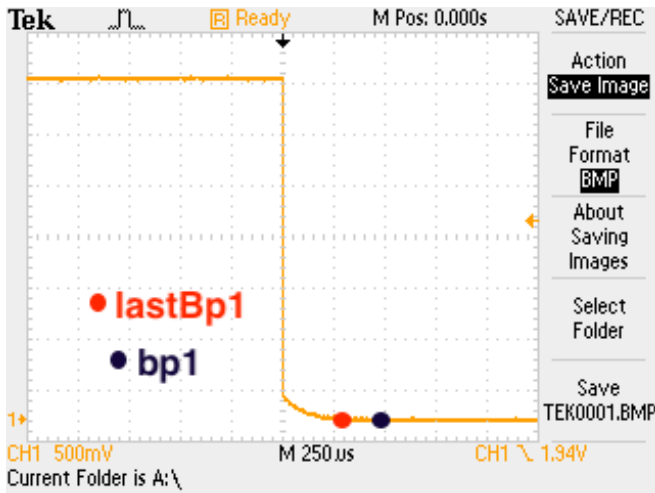


Le test **(bp1 == LOW && lastBp1 == HIGH)** est faux

TP1 - Détection de front sur signal numérique



Le test (**bp1 == LOW && lastBp1 == HIGH**) est vrai



Le test **(bp1 == LOW && lastBp1 == HIGH)** est faux

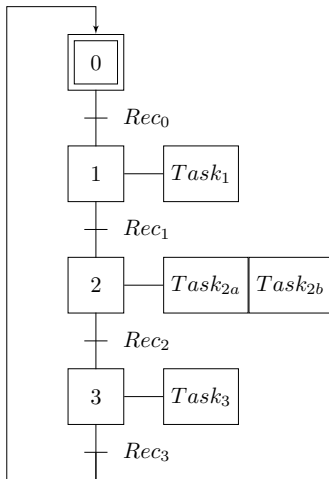
```
1 bp1 = digitalRead(BP1_PIN);
2 if(bp1 == LOW && lastBp1 == HIGH) {
3     //...
4 }
5 lastBp1 = bp1;
```

Détection de front descendant

```
1  if(time - lastBp1Time > 30) { // Echant. tous les 30ms
2      bp1 = digitalRead(BP1_PIN);
3      if(bp1 == LOW && lastBp1 == HIGH) {
4          //...
5      }
6      lastBp1 = bp1;
7      lastBp1Time = time;
8  }
```

Détection de front descendant avec suppression des rebonds

Exemple général de GRAFCET



```
// Data acquisition ...

Rec0 = ...;
Rec1 = ...;
Rec2 = ...;
Rec3 = ...;

if(currentState == state0 && Rec0) currentState = state1;
else if(currentState == state1 && Rec1) currentState = state2;
else if(currentState == state2 && Rec2) currentState = state3;
else if(currentState == state3 && Rec3) currentState = state0;

switch(currentState) {
    case state1:
        task1();
        break;
    case state2:
        task2A();
        task2B();
        break;
    case state3:
        task3();
        break;
    default:
        taskDefault();
        break;
}
// ...
}
// ...
void taskDefault() {
    // ...
}
void task1() {
    //...
}
//...
```

Figure: Exemple général

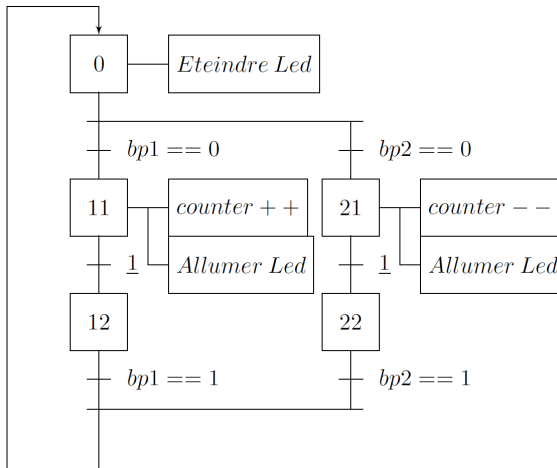


Figure: Prog. 10 - GRAFCET

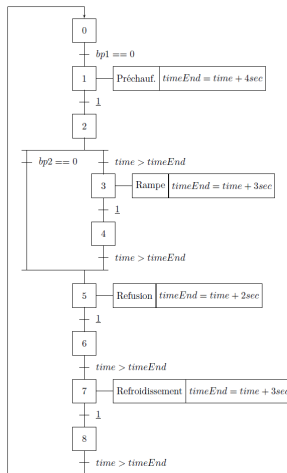


Figure: Prog. 11_1 - GRAFCET

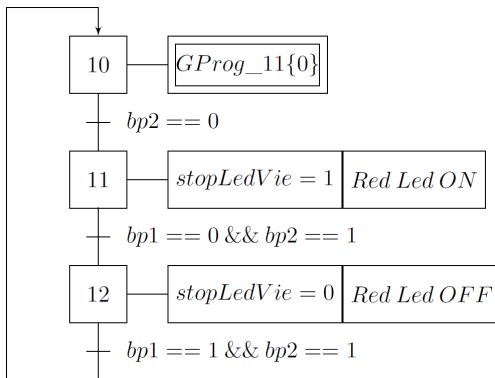


Figure: Prog. 11_2 - GRAFCET

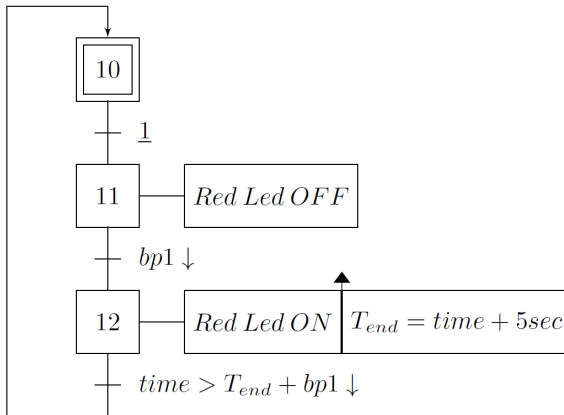


Figure: Prog. 12 - Minuterie arrêt immédiat



La relation qui donne la valeur de l'éclairement E_v en lux en fonction du nombre n délivré par le CAN peut être déduite du graphe $\log/\log \frac{IP_{ce}}{IP_{ce}(1000/x)}$ en fonction de E_v donné dans le TP numéro 2. Selon ce graphe il existe une relation affine entre ces deux grandeurs. On peut donc écrire :

$$\log\left(\frac{IP_{ce}}{IP_{ce}(1000/x)}\right) = a \cdot \log(E_v) + b$$

En prenant les deux points particuliers suivants : $\frac{IP_{ce}}{IP_{ce}(1000/x)} = 10^{-1}$ pour $E_v = 10^2$ et $\frac{IP_{ce}}{IP_{ce}(1000/x)} = 10^1$ pour $E_v = 10^4$, on peut obtenir le système d'équation suivant :

$$\begin{cases} a \cdot 2 + b = -1 \\ a \cdot 4 + b = 1 \end{cases}$$

et en déduire que $a = 1$ et $b = -3$

En utilisant ces valeurs dans la première relation :

$$\log\left(\frac{IP_{ce}}{IP_{ce}(1000/x)}\right) = 1 \cdot \log(E_v) - 3 = \log(E_v) + \log(10^{-3})$$

$$\Rightarrow \log\left(\frac{IP_{ce}}{IP_{ce}(1000/x)}\right) = \log(10^{-3} \cdot E_v)$$

Finalement :

$$E_v = 10^3 \cdot \frac{IP_{ce}}{IP_{ce}(1000/x)}$$

Le courant qui sort du collecteur de Q1 est mesuré aux bornes de R1 : $V_{Light1} = R1 \cdot IP_{ce}$. En remplaçant IP_{ce} dans l'équation précédente on obtient :

$$E_v = 10^3 \cdot \frac{V_{Light1}}{R1 \cdot IP_{ce}(1000/x)}$$

Comme $R1 = 51k\Omega = 51 \cdot 10^3$, la relation précédente devient :

$$E_v = 10^3 \cdot \frac{V_{Light1}}{51 \cdot 10^3 \cdot IP_{ce}(1000lx)} = \frac{V_{Light1}}{51 \cdot IP_{ce}(1000lx)}$$

La tension aux bornes de $R1$ peut être exprimée en fonction du nombre n délivré par le CAN :

$$V_{Light1} = \frac{3,3}{1024} \cdot n$$

Au final :

$$E_v = \frac{3,3}{1024 \cdot 51 \cdot IP_{ce}(1000lx)} \cdot n$$

Si on exprime $IP_{ce}(1000lx)$ en μA :

$$E_v = \frac{3,3}{1024 \cdot 51 \cdot 10^{-6} \cdot IP_{ce}(1000lx)(\mu A)} \cdot n \simeq \frac{63,1893}{IP_{ce}(1000lx)(\mu A)} \cdot n$$

Pour différentes valeurs de $IP_{ce}(1000/x)(\mu A)$, La relation devient :

$$E_v \simeq \frac{63,1893}{16} \cdot n \simeq 3,9493 \cdot n$$

$$E_v \simeq \frac{63,1893}{24} \cdot n \simeq 2,6328 \cdot n$$

$$E_v \simeq \frac{63,1893}{32} \cdot n \simeq 1,9746 \cdot n$$

Que l'on peut utiliser telle quelle dans le code pour le XIAO SAMD21.

TP2 - GRAFCET 4

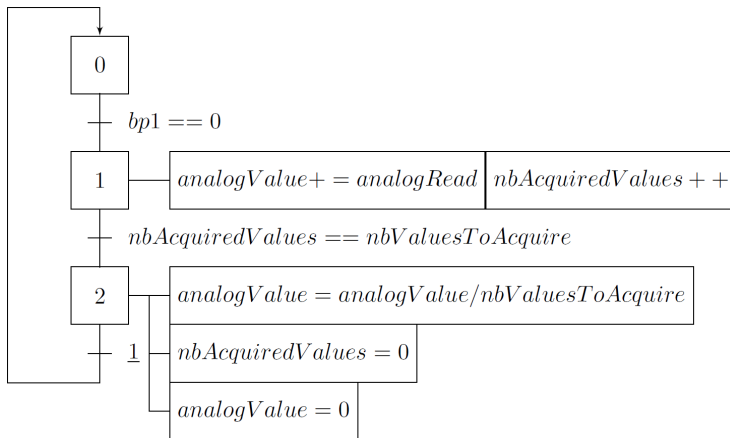


Figure: Prog. 4 - Acquisition de grandeurs analogiques